

Evaluating Agent Skills for Version-Specific Plugin Migration: A Retrospective Study

Beiming Liu¹, Haihao Li^{*2}, Minjie Chen^{*3}, Ning Chen⁴, Yiran Wang⁵, Jiming Ye⁶, Puzhao Zhang⁷,
Tongtao Wang⁸, Sheng Gao⁹, William Jin⁸, Weihao Mu^{10,11}, Chengzhi Liu¹², Yucheng Xia¹³,
Guangren Wang¹⁴, Chaoyang Fan^{15,16}, Changfeng Huang¹⁷, Xunming Lin¹⁸, Yuanjie Shen¹⁹, Qianyu Xiao^{†1}

¹China Financial Certification Authority (CFCA)

²Fudan University

³PetroChina Southwest Oil & Gasfield Company

⁴Jilin University

⁵The Frederick Gunn School

⁶Shenzhen University

⁷Dalian Neusoft University of Information

⁸Independent Researcher

⁹University of Chinese Academy of Sciences

¹⁰Dalian University of Technology

¹¹Jiyin Zhiyuan (Shanghai) Technology Co., Ltd.

¹²Lanzhou University of Technology

¹³Harbin Engineering University

¹⁴Alibaba Cloud

¹⁵Jiangnan University

¹⁶Jiuxiangxian (Beijing) Technology Co., Ltd.

¹⁷Sun Yat-sen University

¹⁸Great Bay University

¹⁹Beijing Information Science and Technology University

Abstract

Agent skills package version-specific maintenance knowledge for coding agents, but a higher diagnostic score does not by itself show that the resulting migration advice satisfies the target version’s contract. We study a shipped plugin-upgrade skill through an archive of 64 reports on 16 static migration tasks, with two attempts per condition and 328 criterion decisions. With the skill, mean recorded reward rises from 93.83 to 98.75, a gain of 4.92 points (95% task-bootstrap interval [0.31, 10.86]); the gain is concentrated in one task, and eight task pairs are at the ceiling. Tracing every decision to its contract domain and reviewing ten reports in depth exposes grading errors that favor either arm; in one, a containment predicate that accepts the parent directory still receives full credit. Executable probes confirm this defect and show that a working teardown repair is excluded only by a narrower lifecycle rubric. Replacing the reviewed decisions keeps the estimate positive (4.61 to 5.39 points) but moves its interval to or across zero. Re-grading all 64 reports with judges from two other model families, without arm labels or prior scores, agrees with the original judge on 91.8% and 95.7% of decisions (weighted $\kappa = 0.64$ and 0.72) and gives gains of 10.63 and 6.09 points. The study contributes a traceable evaluation that connects aggregate reward to contract-level evidence and judge sensitivity; a failure-mode taxonomy coded from the released 61-task incident corpus, in which nearly half the failures are silent and nine misdirect; and concrete review

^{*}Equal contribution.

[†]Corresponding author: xiaoqianyu@cfca.com.cn

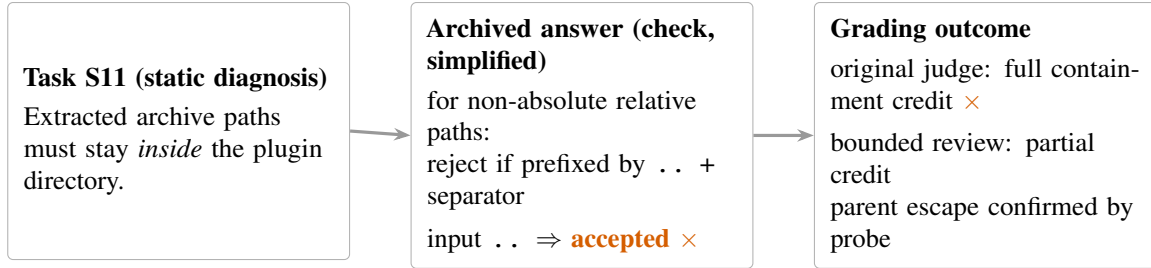


Figure 1: A proposed containment guard accepts the parent directory despite full original containment credit. The archived with-skill answer to task S11 proposes a containment predicate that accepts the literal path `..`, the parent directory, and the original judge still awards full containment credit; bounded review lowers the criterion to partial credit. Sections 4.5 and 5.2 give the full analysis.

checks for migration advice. Executable end-to-end repairs, independent human annotation, and other frameworks are left to future work.

Keywords: software maintenance; plugin migration; agent skills; retrospective evaluation; LLM-as-a-judge

1 Introduction

Software migration requires more than replacing names in source code. API changes routinely break client code (Dig and Johnson, 2006; Raemaekers et al., 2017), and clients often defer updates because migration is costly (Kula et al., 2018). A repair must respect the contract of the target version, distinguish public APIs from internal implementation, and preserve runtime behavior. Plugin systems make these requirements particularly visible: a change can affect loading, authentication, rendering, event dispatch, or process shutdown independently. Advice that sounds consistent with a release note may still recommend an unavailable API or omit a boundary condition. In one archived answer studied below, a proposed path-containment predicate accepts the parent directory, and an automated judge still awards it full containment credit (Figure 1).

Large language models (LLMs) now support a wide range of software-engineering tasks (Fan et al., 2023; Hou et al., 2024), and coding agents resolve repository-level issues by inspecting, editing, and testing code (Yang et al., 2024; Wang et al., 2025b; Xia et al., 2025). An agent skill bundles instructions, reference material, and sometimes executable tools that such an agent loads when relevant (Zhang et al., 2025). Such a package can reduce the effort needed to locate version-specific facts. It can also introduce stale instructions, unnecessary work, or confidence in an incomplete repair. Existing skill benchmarks already demonstrate heterogeneous benefits and overhead (Li et al., 2026; Han et al., 2026; Yang and Ding, 2026). Consequently, the contribution of a maintenance case study cannot rest only on showing that a skill sometimes helps.

The practical question is whether an observed score gain identifies advice a maintainer can act on. An answer can name the correct migration concept while leaving the required operation unavailable, an edge case unhandled, or a runtime condition unmet. Aggregate reward does not show which of these distinctions was checked.

We investigate the shipped plugin-upgrade skill for DSH, linking task contracts to archived answers and the judgments that reward them. The available evidence accumulated during development rather than under one prospectively controlled experiment. We retain this history, distinguish incompatible protocols, and give greatest analytical attention to a later comparison with archived outputs, per-criterion judgments, and execution records. The study asks:

1. RQ1: What changes in recorded reward accompany skill availability in the focal comparison, and how are they distributed across tasks?
2. RQ2: Where does recorded contract credit change, and what do inspected answers show about the adequacy of migration advice and its grading?
3. RQ3: How do resource accounting and grading sensitivity qualify the apparent benefit?

The study makes four contributions.

- An auditable within-configuration comparison that links 64 answers to version-pinned tasks, original criterion judgments, and resource records (Sections 3 and 4.1).
- A two-level account of grading: complete contract-stratified accounting of all 328 decisions, a bounded review that locates judgments missing an operational requirement, and a blinded cross-family re-grading that measures how much the result depends on the judge (Sections 4.2–4.4, 5.2, and 5.3).
- Executable predicate and timer checks that separate an observed defect from an alternative repair excluded by a narrower rubric, and three review questions for maintainers derived from these cases (Sections 4.5 and 6.2).
- A failure-mode taxonomy of plugin-ecosystem breaking changes, author-coded from the 61 incident-derived tasks of the released corpus: nearly half fail silently and nine misdirect, naming the failure classes that aggregate scores and presence guards under-detect (Section 6.3, Appendix B).

The findings concern one plugin framework and static diagnosis; Section 7 discusses validity and Section 8 the studies that would extend them.

2 Related Work

Skills and software engineering. SkillsBench evaluates skills across diverse tasks (Li et al., 2026). SWE-Skills-Bench examines their utility in software engineering (Han et al., 2026). SkillLens studies skill generation and consumption (Huang et al., 2026), while WebDev-Skills-Bench compares skill conditions in web development, including controls for supplied context (Yang and Ding, 2026). These studies establish that skill benefits are heterogeneous and carry resource overhead. We complement their breadth with depth: an artifact-traceable account of version-contract errors and evaluation sensitivity in one maintained plugin-migration setting.

API evolution and migration. Breaking API changes are frequent and costly for clients. Most client-breaking changes in the frameworks studied by Dig and Johnson (2006) are refactorings; about a third of Maven releases, including minor ones, introduce at least one breaking change (Raemaekers et al., 2017); and ecosystems differ in who bears the cost of a break (Bogart et al., 2016). Library developers break APIs mainly to add features and simplify interfaces (Brito et al., 2018), whereas clients lag behind API updates (McDonnell et al., 2013; Kula et al., 2018), and dependency constraints only partly follow semantic versioning (Decan and Mens, 2021). Lamothe et al. (2022) survey this literature. Migration support has progressed from edits mined from migration histories (Xu et al., 2019) to LLM-based migration, from first library-migration results (Almeida et al., 2024) to industrial migrations at Google (Ziftei et al., 2025; Nikolov et al., 2025). VersiCode and CODEMENV benchmark version-controllable code generation and code migration (Wu et al., 2024; Cheng et al., 2025). These studies motivate our focus on version contracts: the target of a migration is defined by what the new version exposes and requires, not by the names that changed.

Coding agents and their evaluation. Surveys document the rapid adoption of LLMs across software-engineering tasks (Fan et al., 2023; Hou et al., 2024). Agent frameworks such as SWE-agent and OpenHands give models tools for navigating and editing repositories (Yang et al., 2024; Wang et al., 2025b), and a simpler localize-repair-validate pipeline is competitive with such agents on SWE-bench Lite (Xia et al., 2025). Evaluations such as SWE-bench and AgentBench emphasize task and environment design (Jimenez et al., 2024; Liu et al., 2024). Our setting combines migration diagnosis with historical executable tasks; the most completely archived comparison consists of static answers, so its reward measures diagnostic advice rather than executable repair success.

LLM judges and measurement validity. LLM judges enable scalable evaluation (Zheng et al., 2023; Chiang and Lee, 2023; Liu et al., 2023), including of code (Zhuo, 2024), and their agreement with human scores has been measured for code translation, generation, and summarization (Wang et al., 2025a). They exhibit position bias (Wang et al., 2024) and favor their own generations (Panickssery et al., 2024), which matters when solvers and judges share a model family. Recent surveys and a large-scale judge study distinguish agreement and consistency from validity (Gu et al., 2024; He et al., 2025; Norman et al., 2026). A panel of judges from different model families reduces single-model bias (Verga et al., 2024); our cross-family re-grading applies this idea as a sensitivity analysis (Section 4.3). For empirical software-engineering studies involving LLMs, contamination, non-determinism, and closed models are recognized threats (Sallou et al., 2024; Wagner et al., 2025). Development exposure is one form of contamination: repeated use of the same task during skill refinement does not create a fresh holdout (Golchin and Surdeanu, 2024). We record each task’s exposure (Section 3.2).

3 Setting and Evidence

3.1 Versioned migration and the intervention

DSH is a plugin-based framework with a marketplace and frequent releases. Relevant changes include renamed exports, authentication boundaries, build-artifact layouts, session representations, and version-routing rules. Tasks specify an initial state, a target contract, permitted operations, and a deliverable. Static tasks request diagnosis; hands-on tasks require a repair in a version-pinned environment. A host cold boot checks liveness, while task-specific probes exercise additional requirements. Neither establishes correctness beyond the checked behaviors.

Upgrade cards record symptoms, causes, actions, version applicability, and provenance. For example, card `DSH-0.1.2-A1-01` concerns removal of the Host APIProxy surface on the 0.1.1-rc.2 to 0.1.2-alpha.1 edge. Correct migration requires locating actual call sites and selecting the corresponding Remote interfaces, rather than applying a client-plane recipe to every plugin. This illustrates why merely mentioning the correct release or card is insufficient.

The shipped skill links inspection, version-corridor analysis, implementation, and verification. Cards can inform both the skill and the benchmark. Thus the comparison is an open-book evaluation of the complete package, including access to facts, rather than an isolated test of procedural reasoning. Historical repository states differ, so the intervention is not assumed byte-identical across configurations.

3.2 Task construction and exposure

Contributors turn migration incidents and version changes into cards, fixtures, instructions, reference solutions, and graders. Contributor self-check and maintainer review constitute development quality control, not independent blinded annotation. The repository’s exposure ledger distinguishes feedback used to refine the

skill, grader development, and uncleared exposure. We make no claim that the evaluated tasks form an independent holdout.

An immutable earlier snapshot contains 23 tasks (10 static and 13 hands-on). Task identifiers carry a type prefix (S for static, M for mixed-mode, H for hands-on), and this snapshot contains S1–S8. The static pool was later extended to 22 tasks (S1–S22); the historical GLM configurations scored that pool in full, and the focal comparison draws its recorded selection of 16 static tasks from it. This inventory is not the denominator of every experiment. Historical configurations use 56, 23, 21, or 22 scored tasks. Repeated attempts are nested within tasks; they do not increase the number of independent migration problems. Related tasks may also share incident sources, limiting task-level uncertainty estimates.

3.3 Evidence tiers

We distinguish three evidence tiers. The focal comparison answers the research questions; the other tiers document the available context.

1. Focal archived comparison: 16 tasks, two conditions, and two repeats, with 64 reports, original verdicts, scores, selection and execution records, and an offline integrity audit.
2. Historical context: five within-configuration paired summaries, with heterogeneous task pools, aggregation, grading, and raw-artifact coverage.
3. Supplementary evidence: a newer 16-task Qwen comparison with scored records but without archived original answers and judgment reasons, plus three GLM-5.3 rounds with a change of judge in round three. Neither is pooled into the focal estimate.

4 Method

We report the study as a retrospective single-case study (Runeson and Höst, 2009); in the terms of Stol and Fitzgerald (2018), it is a field study that favors realism over generalizability, such studies are covered by the case-study standard of the ACM SIGSOFT empirical standards (Ralph et al., 2020). Figure 2 summarizes the focal study design end to end: a seeded selection from the static task pool, paired execution under two conditions, criterion-level grading, and the three analyses that answer the research questions. The subsections below follow this flow.

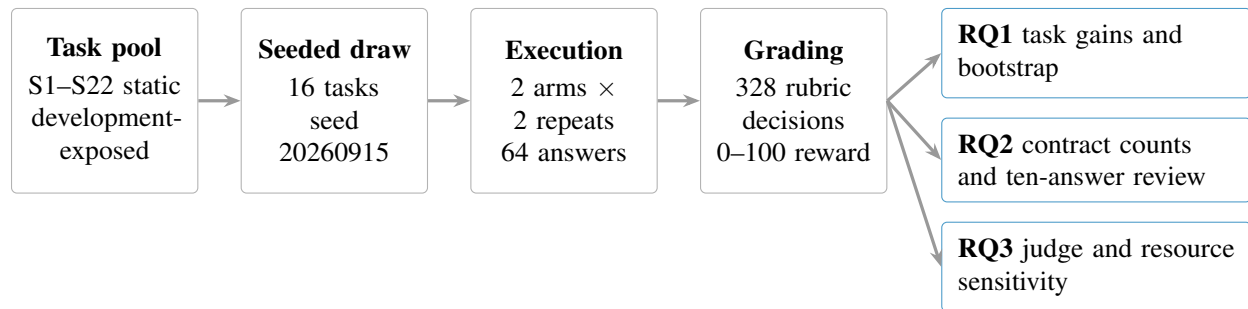


Figure 2: Overview of the focal study design. A seeded stratified draw selects 16 of the 22 static tasks; two arms with two repeats produce 64 archived answers, graded by the report-judge-v2 rubric into 328 criterion decisions. Three analyses consume this archive: the focal paired estimand (RQ1), complete contract-stratified accounting and a bounded ten-answer review (RQ2), and grading-sensitivity plus resource accounting (RQ3).

4.1 Focal comparison and estimand

The focal run is the archived GLM-5.3-Flash unified S16 comparison; “S16” here names the archived configuration of sixteen selected static tasks, not the task identifier S16. The model label is the recorded gateway label, not an independently verified served-weight identity. Each task has two no-skill and two with-skill outputs, run as subagent sessions under the ZCode agent host. The conditions share a prompt template, but workspace paths differ. Two focal tasks (S5 and S6) carried run-time brief wording that described the diagnosis as closed-book, which did not anticipate a mounted skill; the briefs have since been amended to acknowledge harness-mounted materials, and the archived task packets preserve the run-time text under hash.

The selection script draws 16 of the 22 static tasks S1–S22 using seed 20260915, stratifying by problem family and fixture-size tercile with proportional quotas. Balance constraints require at least two hard tasks, two Chinese-prompt tasks, and two large-fixture tasks. The selector reads metadata and fixture sizes rather than scores; it excludes tasks S3, S4, S7, S8, S10, and S16. The selected set contains five static-contract diagnosis, seven runtime/client API, three release/install, and one Cordis profile task (Cordis is the underlying plugin-runtime dependency of DSH); 14 prompts are English and two are Chinese. The draw is reproducible, and it is drawn from the development-exposed pool described in Section 3.2.

The schedule alternates arm order in task blocks and reverses it in round two; the recorded execution order is retained. Four pilot cells on excluded tasks S4 and S10 are not included in the 64 formal cells. Formal execution used no hard wall-time cap, and all 64 cells produced scored reports.

Let y_{tar} be the original score for task t , arm a , and repeat r . We calculate $\bar{y}_{ta} = (y_{ta1} + y_{ta2})/2$, $d_t = \bar{y}_{t,\text{skill}} - \bar{y}_{t,\text{no}}$, and $\hat{\Delta} = 16^{-1} \sum_t d_t$. Thus tasks receive equal weight. The focal percentile bootstrap (Efron and Tibshirani, 1993) resamples paired tasks 10,000 times (mulberry32, a small deterministic pseudo-random number generator, seed 20260915). It quantifies variability over the observed tasks under resampling assumptions, not uncertainty over arbitrary repositories or model versions.

The report-judge-v2 rubric, the harness’s criterion-level grading protocol, assigns a weight to each criterion. A pass receives its full weight, partial receives half, and fail or missing receives zero; declared triggered caps can limit the total. Criteria and frozen reference excerpts are supplied to the judge, which returns decisions and reasons rather than a total. The deterministic scorer calculates the 0–100 reward. Proposed verification can satisfy a static task; describing a test does not establish that it was executed.

The same model family supplies semantic evaluation. The run records describe arm-blind staging with one report per judgment; our later review was not blind. The archive retains original verdicts, including historical transport metadata; corrected provenance records distinguish harness-declared judge identity from independently observed provider identity. Scores are recomputed from rubric decisions. An integrity audit checks all 64 report hashes and all 328 original criterion decisions. Two stored reports require reversal of documented relative-link rewriting to reproduce their original hashes; this transformation is explicit rather than silently ignored.

4.2 Bounded output review

The initial targeted review was AI-assisted and non-blind, separate from the original verdicts. An earlier purposive pass covered five complete answers selected for prominent effects or operational concerns. A subsequent selection was committed before reading the additional full reports: paired answers for a positive task (S6), a negative task (S18), and a zero-change task (S12), at fixed repeats. One answer overlapped with the earlier pass. The combined scope is ten unique answers and 56 criteria.

The initial reviewer had access to task rubrics, reports, and earlier judgments, and selection used known outcomes. Multiple contributing plugin authors, who are domain experts for these plugins rather than independent annotators, subsequently performed the human checking and reported broadly similar

assessments; the later repository revision identifies them as non-blind reviewers. The ten archived answers retain the same 56 decisions and replacement scores across these revisions. No separate per-reviewer scoring sheet or item-level confirmation was supplied, so revision stability is not interpreted as measured human–AI agreement or independent blinded validation.

The provenance ledger retains both the initial AI-assisted analysis and the reported human follow-up. Selection, report hashes, criterion-level reasons, and replacement scores are recorded. We keep original results as the primary recorded endpoint and show both the earlier and the combined replacement sensitivities. Replacements affect only reviewed answers; unreviewed scores are not presumed correct.

4.3 Cross-family judge panel

Because the focal solver and judge share a model family, we re-judged all 64 focal reports with judges from other families under a protocol and analysis plan written before any panel verdict existed (Appendix D). Each item contains the unchanged report-judge-v2 system prompt and judge input (task, rubric, frozen excerpts, citation audit, and report), and nothing about arm, repeat, original verdicts, or review outcomes; item identifiers are opaque hashes and the unblinding key is hash-pinned.

Claude Opus 5.5 (`claude-opus-5-5`) ran as 16 fresh agent sessions of four items each, so that no session saw two answers to the same task. GPT-5.5 (`openai-gpt-5.5`) processed the 64 items sequentially in one isolated Codex CLI session. Neither judge saw the other’s verdicts. Appendix D records session provenance.

Blinding is limited to supplied labels and prior outcomes. Arm is only partly concealed: 18 of 32 with-skill reports mention the skill in their own text, as they did for the original judge. The committed reports include the documented relative-link rewriting noted above.

Following the analysis plan, we measured criterion-level agreement on the credit scale (pass, partial, fail or missing) with linearly weighted Cohen’s κ (Cohen, 1960, 1968) and recomputed the focal estimand under each judge and under the per-answer judge mean. Agreement with the author review is reported as a reference point; that review started from the original decisions and is not ground truth. The panel measures sensitivity to judge configuration; human validation is discussed in Section 8.

4.4 Complete contract-stratified analysis

We map each of the focal tasks’ existing rubric criteria to one primary domain: version/release applicability; API/data/ownership; lifecycle/ordering/deployment; safety/boundary handling; failure attribution; or evidence/verification/provenance. The mapping is retrospective and fixed before generating the domain table. Multi-part criteria receive one primary label to avoid double counting, so these categories are an analytical organization of the rubric rather than an independently validated failure taxonomy.

Every formal report is linked to its task packet and original criterion verdicts through stored hashes. The analysis accounts for all 64 reports and all 328 criterion decisions, including original successes and shortfalls. We report full-credit counts with per-arm criterion denominators and distinct task counts. The counts are recorded judge outcomes; semantic re-grading is the role of the bounded review and the judge panel. Repeats are nested within tasks, tasks can contribute to multiple domains, and we make no domain-level significance claims.

4.5 Bounded mechanism and robustness checks

After case selection and the original analysis, we added offline checks under a recorded retrospective protocol. For S11, we extracted the archived proposed containment function, removing only TypeScript annotations, and exercised seven declared lexical inputs under each of Node’s POSIX and Windows path algorithms.

Focal endpoint	No skill	Skill	Δ	95% task-bootstrap interval
Original judgments	93.83	98.75	4.92	[0.31, 10.86]
Earlier S11 replacement	93.83	98.44	4.61	[−0.23, 10.63]
All reviewed replacements	93.05	98.44	5.39	[0.00, 11.80]

Table 1: Focal 16-task comparison. Original scores remain unchanged in the archive. Replacement rows are partial-review sensitivity analyses, not validated estimates from a fully regraded dataset.

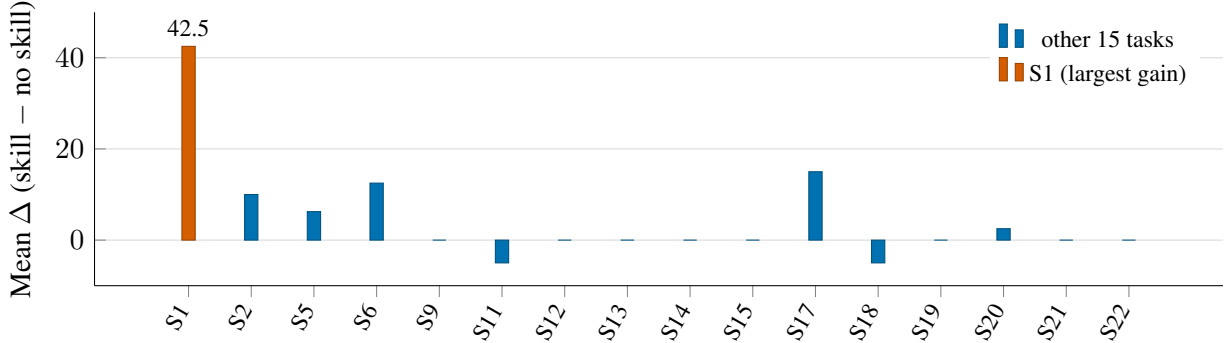


Figure 3: Task-level mean reward changes in the focal comparison are concentrated in one task and bounded by ceiling effects. S1 alone gains 42.5 points, eight task pairs score 100 in both arms (zero change), and two tasks decline by 5 points. Per-task scores are in Table 5.

Parent and ancestor escapes, ordinary and dotted descendants, prefix siblings, and platform-specific controls distinguish the observed defect from ordinary valid behavior. The probe exercises the predicate itself, not an HTTP route, native Windows filesystem operations, or symlink resolution.

For S18, the fixture describes a timer planner but does not contain its implementation. We therefore reconstructed a minimal self-rearming timer in separate Node child processes. Five conditions compare referenced timers, cleanup that is not invoked, explicit unmount with cancellation, unreferencing every timeout, and unreferencing only the first timeout while another handle initially keeps the process alive. Each child emits a readiness marker; a parent watchdog terminates children still alive 700 milliseconds later. Natural exit is distinguished from watchdog termination. These are mechanism checks with controls, not executions of the original plugin or new scored solver attempts.

For the original and both existing replacement endpoints, we also enumerate all $2^8 = 256$ sign assignments of the eight nonzero paired task differences. The two-sided tail fraction counts assignments whose absolute sum is at least the observed absolute sum, including ties. This is a descriptive sensitivity under independent, sign-exchangeable task differences; the deterministic arm schedule does not justify a causal randomization test. We additionally report every leave-one-task-out mean and both repeat-wise means. None of these endpoints is selected by its significance.

5 Results

5.1 RQ1: Recorded reward changes

In the focal comparison, mean original reward increases from 93.83 to 98.75 (Table 1). Six tasks improve, two decline, and eight have zero change (Figure 3); all eight zero-change pairs are at 100 in both arms. S1 has a 42.5-point task-level gain. Removing it reduces the overall gain to 2.42 points across the remaining 15 tasks.

Primary domain	Tasks	Criteria/arm	No skill	Skill
Version and release applicability	6	12	8	12
API, data and ownership contracts	13	46	35	45
Lifecycle, ordering and deployment	9	26	24	25
Safety and boundary handling	5	10	10	10
Failure attribution	11	32	32	32
Evidence, verification and provenance	14	38	31	36

Table 2: Complete contract-stratified analysis of the focal 64 reports. The final two columns count original full-credit criterion decisions; they are not independently validated correctness counts. Criteria/arm includes both repeats. Multi-contract criteria receive one retrospective primary label; tasks can contribute to multiple domains.

The original Wilcoxon signed-rank approximation (Wilcoxon, 1945) gives $p = 0.0797$ with eight nonzero pairs. The corresponding exhaustive sign-enumeration tail fraction is $20/256 = 0.0781$. Repeat-wise mean gains are 4.53 and 5.31; all 16 leave-one-task-out means range from 2.42 to 5.58. Every variant keeps the direction positive. Because eight pairs are at the ceiling and one task contributes most of the gain, we read the focal estimate as a descriptive association.

The focal estimate describes this configuration and task set. Historical and supplementary configurations, which use different task pools, hosts, and judges, are reported separately in Appendix C.

5.2 RQ2: Contract failures and grading disagreements

Figure 4 displays full-credit proportions across the full cohort; Table 2 gives the counts and task coverage. Original full-credit counts rise from 8 to 12 out of 12 version-related decisions and from 35 to 45 out of 46 API/data/ownership decisions per arm. Failure attribution has full credit in both arms (32/32), as does the safety/boundary domain (10/10). These counts describe the original judge; the reviewed S11 containment defect demonstrates why an all-full-credit domain cannot be called verified safe. Domains contain different tasks and weights, so their percentages are not a difficulty ranking or an estimate of a causal skill mechanism.

The bounded targeted review finds three criterion disagreements in two of ten answers. The sample is purposive, so these disagreements identify concrete weaknesses in the recorded measurement rather than an error rate.

Information access and public API boundaries. S1’s reviewed answers retain their original 55 and 100 scores. Part of its rubric rewards mapping to supplied migration cards. The large gain therefore includes access to organized facts and identifiers, and cannot isolate a procedural reasoning benefit. In S6, the no-skill answer correctly recommends removing obsolete defensive code but incorrectly attributes informational-event handling wholly to the host. It also recognizes a public append API gap while recommending an unqualified append route. Review changes two criteria and lowers this answer from 87.5 to 62.5. The paired skill answer retains 100 by distinguishing producer responsibilities, retention, and unavailable public surfaces. The resulting sensitivity increases estimated skill benefit, illustrating that grading errors can favor either arm. Additional full-text inspection of S6 repeat 2 finds the same platform-only interpretation in the no-skill answer, whereas both skill answers distinguish retention from a supported producer seam. This is repetition within one task, not a second independent incident; no additional score replacement is imposed in the fixed ten-answer sensitivity.

Containment boundaries. In S11, a with-skill answer proposes a relative-path check equivalent to accepting an empty relative path or any non-absolute path that does not start with two dots followed by a separator.

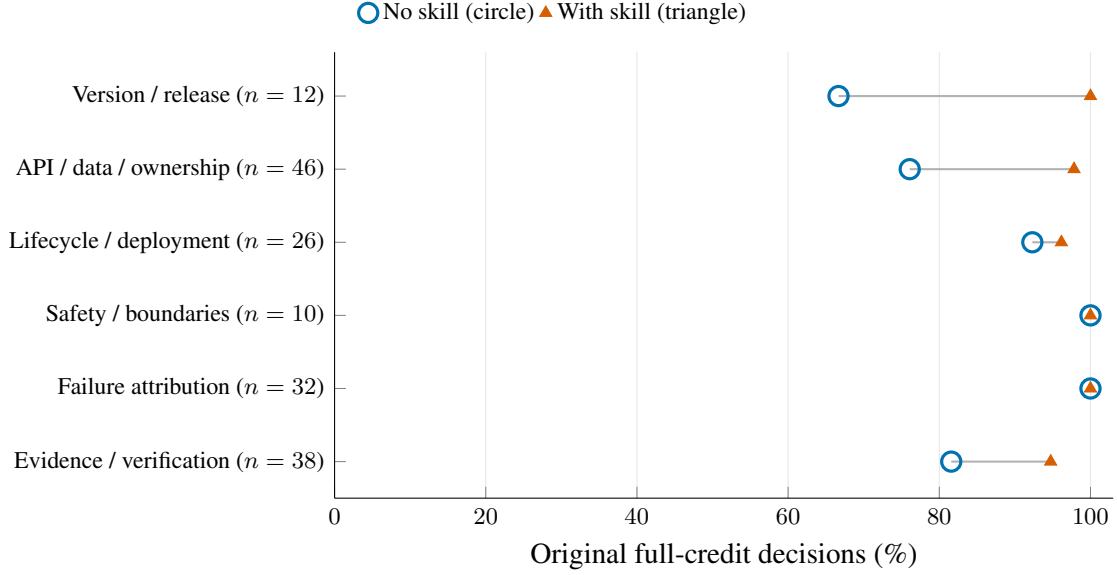


Figure 4: Original full-credit proportions rise in four contract domains and remain at 100% in two. Points summarize all 328 criterion decisions across 64 reports; n is the number of criterion decisions per arm, including both repeats. Circles and triangles overlap at 100% for safety/boundaries and failure attribution. These are finite-cohort descriptions of the original judge, not correctness rates or independent samples; the reviewed S11 boundary defect remains despite full original boundary credit.

The exact relative path `..` passes this predicate, even though it denotes the parent directory. The extracted predicate accepts the parent in both path algorithms, contradicting the declared containment expectation in two of 14 diagnostic inputs; the other 12 control inputs behave as expected. Review lowers the containment criterion from full to partial credit and the answer from 90 to 80. This confirms a defect in the proposed guard; whether it is exploitable depends on the surrounding route, which the probe does not execute.

Cleanup versus process liveness. For S18 repeat 2, the no-skill answer retains 100 and the skill answer retains 90. The skill answer proposes explicit host teardown, while the rubric requires timers that do not keep a still-mounted probe host alive. Our child-process reconstruction makes the distinction observable: a referenced chain and an uninvoked cleanup callback remain alive until the watchdog stops them; both explicit unmount with cancellation and unreferencing every timeout exit naturally. Unreferencing only the first timeout is insufficient when an initially live companion handle allows a referenced successor to be scheduled.

Thus explicit teardown is a functioning alternative in the reconstruction when the host performs it. The retained partial credit measures noncompliance with the rubric’s narrower mounted-host requirement, not proof that the proposed cleanup can never repair a CI hang. The instruction asks for a one-line hang fix, whereas the rubric specifies timer unreferencing. This difference exposes sensitivity to how acceptable repairs are defined. We preserve the original and prior reviewed scores; the new mechanism check does not silently create another replacement endpoint.

Ceiling and a retained partial score. Both reviewed S12 answers retain 100: they connect a Windows file lock to the native host process, distinguish the recorded release channels, and prescribe the pinned installation and restart sequence. This zero-change pair illustrates a ceiling case rather than evidence that the skill was

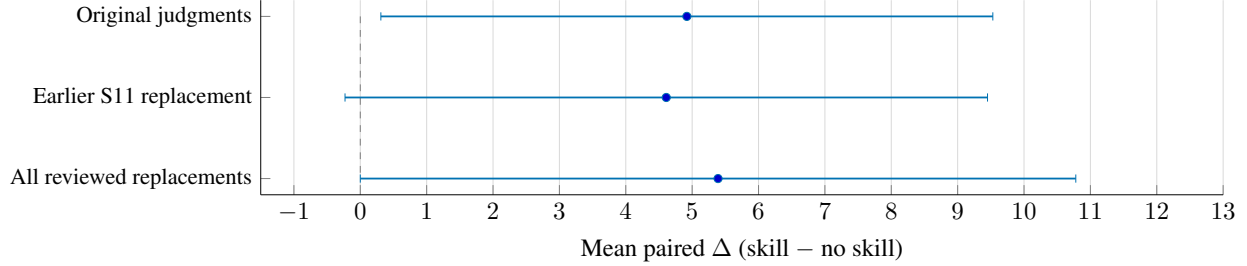


Figure 5: Grading sensitivity of the focal gain. Replacing reviewed criterion decisions moves the mean change between 4.61 and 5.39 points; the earlier S11-replacement interval crosses zero and the all-reviewed interval touches it (dashed line), so the positive direction survives while the strength of evidence depends on grading decisions. Whiskers are 95% task-bootstrap intervals.

Judge	no-skill → with-skill	Δ	95% CI	vs. original	vs. author
GLM-5.3-Flash (original)	93.83 → 98.75	+4.92	[+0.31, +10.86]	–	53/56
Claude Opus 5.5	87.19 → 97.81	+10.63	[+3.44, +19.14]	91.8% / 0.64	51/56
GPT-5.5	93.28 → 99.38	+6.09	[+1.56, +11.09]	95.7% / 0.72	48/56
All-judge mean	91.43 → 98.65	+7.21	[+1.98, +13.46]	–	–
Cross-family mean	90.23 → 98.59	+8.36	[+2.66, +14.96]	–	–

All 64 focal reports (328 criterion decisions) re-judged without supplied arm labels or prior scores under the unchanged report-judge-v2 prompt and rubric. “vs. original”: exact credit-level agreement / linearly weighted κ with the original GLM decisions. “vs. author”: exact agreement with the 56 decisions of the non-blind plugin-author review (10 answers), which is a reference, not ground truth. Between panel judges: claude-opus-5-5 vs openai-gpt-5.5: 90.5% exact, $\kappa_w = 0.57$. All judges are LLMs; this is not human validation.

Table 3: Sensitivity of the focal paired endpoint to judge configuration. Estimand, bootstrap, and seed as in Section 4.1; the original row is the primary recorded endpoint.

consumed or unnecessary in all contexts. The previously reviewed S17 no-skill answer retains 80 because recognizing onboarding requirements does not repair an incorrect injection descriptor.

5.3 RQ3: Sensitivity and resource use

Replacing only the reviewed S11 score changes the focal estimate to 4.61 points and moves its interval across zero (Figure 5). Replacing all reviewed scores changes it to 5.39 points with a lower endpoint at zero. The direction remains positive, but the strength of evidence depends on grading decisions. The exhaustive sign-enumeration tail fractions are $34/256 = 0.1328$ for the S11-only replacement and $30/256 = 0.1172$ for all reviewed replacements, compared with 0.0781 originally. Neither the larger replacement estimate nor the original positive interval should be selected in isolation.

Figure 6 compares complete judge re-grades and their per-report means; Table 3 gives the exact estimates and agreement statistics.

Claude agrees with the original GLM judge on 301 of 328 criterion credit levels (91.8%, weighted $\kappa = 0.64$); GPT agrees on 314 (95.7%, $\kappa = 0.72$). Claude and GPT agree with each other on 297 (90.5%, $\kappa = 0.57$). In the conventional bands of Landis and Koch (1977), agreement with the original judge is substantial for both panel judges and moderate between them. Their answer scores match the original in 45 and 53 of 64 cases, respectively (Table 3).

The judges differ in both direction and extent of re-scoring. Claude assigns lower credit than GLM to 26 decisions and higher credit to one, with 23 of those reductions on no-skill answers. GPT’s departures

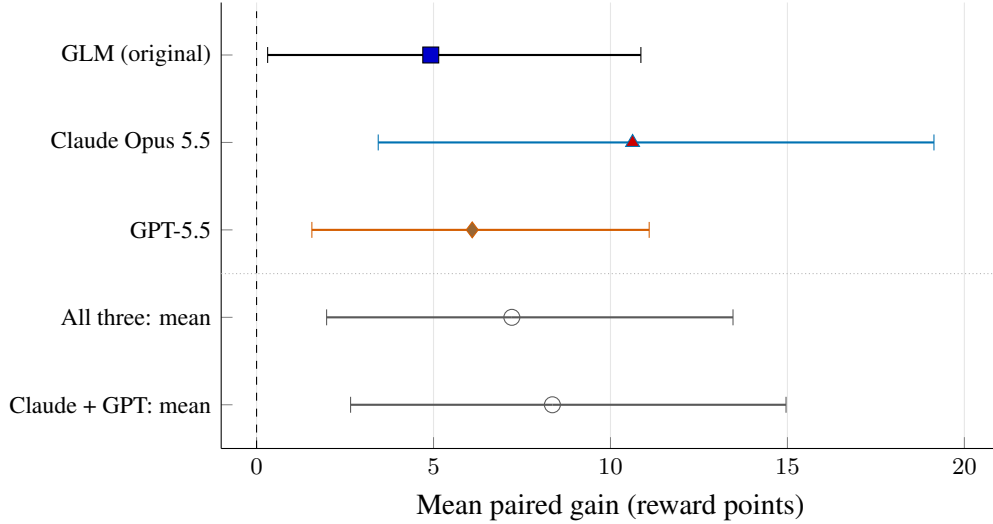


Figure 6: All three judges retain a positive mean gain, while its size depends on the judge. Filled markers show the original GLM score and the two complete LLM re-judgments of the same 64 reports; open markers below the dotted separator average judges within each report and are not additional independent judges. Whiskers are 95% percentile intervals from 10,000 paired-task bootstrap samples over the same 16 tasks (seed 20260915). All rows share the original rubric and reports; these intervals describe task resampling, not uncertainty over judge populations.

are smaller and mixed: five lower-credit and nine higher-credit decisions. Re-graded gains are 10.63 [3.44, 19.14] for Claude and 6.09 [1.56, 11.09] for GPT, versus the original 4.92 [0.31, 10.86]. The per-answer mean across all three judges gives 7.21 [1.98, 13.46]; the mean of the two cross-family judges gives 8.36 [2.66, 14.96].

Agreement does not establish accuracy. The original judge awards full credit to 300 of 328 decisions (91.5%), so high exact agreement must be read alongside the chance-adjusted weighted κ . Against the 56 decisions in the non-blind author review, GLM, Claude, and GPT match 53, 51, and 48; that review was seeded by GLM’s decisions and is no gold standard. Both cross-family re-judgments preserve a positive descriptive direction, but the magnitude varies, and the judges share a rubric whose validity is not established by their agreement. The original judgment remains the primary recorded endpoint.

The 64 formal execution records contain 3,185,993 no-skill and 12,320,379 skill `subagent_tokens`, a ratio of 3.87. The corresponding sums of recorded task durations are 9,446 and 11,450 seconds, a ratio of 1.21. The token field does not separately define input, output, and cache accounting, so it is not a billing-cost measure. Summed task durations are not end-to-end wall-clock time under overlapping execution. Pilot records are excluded.

Together, these results answer RQ3 with a conditional assessment: higher recorded reward accompanies greater recorded token use in the focal configuration. The bounded review’s replacement intervals touch or cross zero, whereas both label-blinded LLM re-judgments retain positive intervals under the specified task bootstrap. These analyses probe different sources of grading sensitivity and do not establish a single corrected score. The archive supports neither a monetary cost-benefit estimate nor a claim that the added resource use produces more successful repairs.

6 Discussion

6.1 What contract inspection adds to aggregate reward

The focal result has two complementary interpretations. Skill availability is associated with a higher mean recorded score, with much of the gain concentrated in one task and no room for improvement on eight others. Inspection then asks whether the rewarded advice meets the particular requirement that makes a migration usable. This second question matters even when the average improves: S11 receives full containment credit for a predicate that accepts the parent directory, whereas S18’s partial credit depends on requiring exit without host teardown. The mechanism check shows that teardown can also permit exit when performed. The cases distinguish a missed predicate defect from a narrower choice of acceptable lifecycle repair.

Full-cohort accounting and targeted inspection answer different questions. The former shows where the original judge assigned credit across every focal answer, avoiding a results table built solely from selected successes or failures. The latter supplies counterexamples with reasons, but cannot estimate how often such defects occur. For example, the safety/boundary domain has full original credit in both arms even though the S11 check fails a concrete boundary case. Neither the domain count nor the selected defect alone characterizes the reliability of the whole dataset.

6.2 Review questions grounded in the observed cases

The cases motivate three checks for maintainers reviewing migration advice. For each proposed API action, verify that the target version exposes a callable public surface and identify the component responsible for producing or retaining the data (S6). For a boundary predicate, test the exact boundary value as well as ordinary descendants; the parent path itself supplies a counterexample in S11. For lifecycle advice, state the required runtime condition separately from its suggested cleanup action; releasing a component resource and allowing process exit are different requirements (S18).

These checks can be applied to an individual answer without relying on its total score. Their evidence differs in kind: the S11 predicate was executed directly, S18 was examined through a minimal timer reconstruction with positive and negative controls, and S6 is a static assessment against the recorded contract. They are review questions transferable to other projects; how often the underlying defects occur there, and whether building the checks into an agent or grader improves outcomes, are open questions (Section 8).

6.3 A failure-mode taxonomy for plugin-ecosystem breaking changes

The 65 benchmark tasks were not sampled from an abstract generator: 61 encode concrete incidents, most drawn from a maintained plugin fleet crossing the same host releases, and four are procedural controls. Coding each incident task by the loudness of its first observable failure (silent, misdirected, loud) and by its blast radius (plugin-local, co-tenant, system) yields a taxonomy that explains why contract-level inspection, rather than aggregate scoring, is the right evaluation unit (Table 4; the full per-task coding is Table 6).

Nearly half of the incident tasks (28 of 61) fail silently: no exception, no console entry, and in the S23 pattern a four-check compatibility guard still passes because the removed field sits inside a snapshot payload rather than a service member. The largest single cell — silent and plugin-local, 26 tasks — is precisely the class that presence guards, crash logs, and aggregate reward all under-detect. Nine tasks are misdirected: the observable failure implicates the wrong component, as when one plugin’s render-time throw unmounts an innocent co-tenant through a shared error boundary (S15, S24) or an in-place upgrade surfaces as file-lock errors (S12). Misdirected failures break the symptom-to-defect mapping that both human maintainers and LLM judges rely on, and they are where incorrect-but-plausible migration advice is hardest to catch. Shared-radius failures (co-tenant or system, nine tasks) are rare but dominate attribution errors.

	plugin-local	co-tenant	system
silent	26 (S3, S6, S7, S9, S14, S18, ...)	1 (H19)	1 (H13)
misdirected	6 (S8, H1, H2, H4, H6, H24)	1 (H8)	2 (S12, S16)
loud	20 (S4, S11, S13, S17, M1, M2, ...)	2 (S15, S24)	2 (S20, S22)

Table 4: Failure-mode taxonomy of the 61 incident-derived tasks (of 65 total; the rest are procedural controls), coded by first-observable-failure loudness and blast radius. Author-coded from task provenance; single-coder.

The taxonomy is author-coded from task provenance (single-coder, not independently annotated), so we treat the cell counts as descriptive structure rather than estimated frequencies. Its role is interpretive: it names the failure classes that make a contract-stratified audit necessary and gives task authors a coverage target — a migration corpus that only exercises loud, plugin-local breaks would systematically overstate what diagnostic scores measure.

6.4 Implications for evaluating maintenance skills

A maintenance-skill evaluation should state which endpoint it measures: locating applicable knowledge, supplying contract-consistent advice, or completing a verified repair. The focal archive measures graded static advice. S1’s card-mapping reward also captures access to organized facts (Section 5.2). Reporting task-level changes and the contract behind each selected shortfall makes these distinctions visible without treating the rubric as a substitute for execution.

In this archive, retaining original verdicts alongside explicit alternative scores is necessary to interpret the result. Correcting the reviewed S6 judgment increases estimated skill benefit, while correcting S11 reduces it. Thus review should inspect both arms and preserve disagreements that strengthen or weaken the apparent effect. Our replacement analyses demonstrate this sensitivity locally. S18 additionally shows why a grader should state whether alternative repairs are acceptable or whether a particular lifecycle condition is mandatory.

The historical ordering does not establish a capability-response curve; model behavior does not always improve monotonically with scale (McKenzie et al., 2024). Baseline reward shares mathematical components with the gain and is bounded by the same ceiling; model comparisons also mix infrastructure, graders, task exposure, and skill versions. The present contribution is an evidence-linked maintenance case study; claims about an inverted U, production reliability, or cost effectiveness would need dedicated designs and endpoints.

7 Threats to Validity

We organize threats following Wohlin et al. (2012) and consider the LLM-specific threats identified by Sallou et al. (2024) and Wagner et al. (2025).

Construct validity. Static diagnostic reward is not functional repair success. Some criteria reward card mappings or metadata. Keyword and semantic graders measure different things, and numeric normalization does not harmonize their constructs. The focal solver and judge share a model family. The two cross-family panel judges (Section 4.3) preserve the positive direction but yield gains of 6.09 and 10.63 points, compared with the original 4.92; all three are language models sharing one prompt and rubric, so agreement does not validate the rubric itself. The single GPT judging session may carry information between sequential items, unlike the original separate calls. Family and session structure are confounded in this panel, and each configuration was judged once; test–retest reliability is unmeasured.

The initial non-blind AI-assisted review is useful for finding counterexamples but does not certify all scores. The human follow-up was performed by contributing plugin authors rather than independent annotators, without a separate item-level scoring dataset, precluding an inter-rater agreement estimate.

Attribution to the skill’s organization remains unverified. No raw-document or generic-procedure control arm was run, so the comparison separates skill availability from absence but cannot attribute the benefit to the skill’s procedural organization rather than its bundled facts (Section 3.1). Full-cohort contract tabulation inherits the original judge decisions and should not be confused with complete revalidation. The S18 instruction and rubric differ in how narrowly they specify the hang repair; compliance with the latter is not equivalent to the absence of other functioning repairs.

Internal validity. The data are retrospective, and contributors developed the skill and tasks in an overlapping workflow. Historical selections are not assumed random, and the focal seeded draw does not remove development exposure. Historical execution budgets, environment failures, and protocols differ. Original model labels rely on recorded infrastructure declarations. The review selection was frozen before additional full-output inspection, but after outcomes were known; it is not preregistration of the underlying experiment. The agent host did not isolate arms in separate containers, and workspace paths differ between arms.

Statistical conclusion validity. There are only 16 focal tasks and two repeats per arm, which is few for evaluating stochastic agents (Arcuri and Briand, 2011). Eight task pairs are at the ceiling. Task-level resampling treats tasks as independent even when incident sources may overlap. No family-clustered analysis is reported (Section 8). The selected review does not estimate population error, and partial replacements leave unreviewed scoring uncertainty unresolved. Intervals, signed-rank approximations, and leave-one-out results are exploratory descriptions of the recorded data rather than confirmatory tests of a universal effect.

External validity and reproducibility. The focal tasks concern one plugin framework and static reports. Historical task inventories and artifacts have unequal coverage. The newer Qwen outputs and grading reasons are unavailable in the repository, limiting replication of its endpoint. Live models and dependencies can change even with pinned task sources. The archive supports deterministic recalculation of the focal scores and reported sensitivities, not a guarantee of identical future model outputs. Mechanism probes are restricted to an extracted lexical predicate and a reconstructed timer; they do not execute complete agent-produced migrations, and the bounded timer observation is not a production liveness guarantee.

8 Future Work

The archive and tooling released with this study make several follow-up studies direct to run.

Independent human validation. The repository already contains an outcome-blind protocol for grouping tasks by incident family with two independent annotators and third-party adjudication (`paper/audit/task-annotation-v1`). Executing it would support family-clustered uncertainty estimates. A blinded human re-grading of a random sample of criterion decisions would calibrate the LLM judges against human judgment and yield an inter-rater agreement estimate.

Attribution and executable endpoints. Raw-document and generic-procedure control arms would separate the value of organized facts from that of the skill’s procedure. Running agent-produced repairs end to end in version-pinned hosts would replace diagnostic reward with repair success, and would test whether the S11 and S18 mechanisms matter in deployed plugins.

Judge reliability. Repeated judging of each configuration, with model family and session protocol varied independently, would measure test–retest reliability and separate the two factors confounded in the present panel.

Prospective replication and transfer. A temporal-holdout split and its execution plan are specified in the repository (`benchmark/holdouts`); running it, and applying the same protocol to other plugin frameworks, would test whether the effect and the review questions of Section 6.2 transfer beyond this setting, and whether adding those questions to an agent or grader improves outcomes.

9 Conclusion

In an archived 16-task comparison, making a plugin-upgrade skill available raised recorded reward by 4.92 points, with the gain concentrated in one task and bounded by ceiling effects. The more consequential finding concerns how such gains should be read. API availability and responsibility, exact boundary coverage, and process liveness cannot be inferred from a plausible recommendation or an aggregate score; they have to be checked at the contract the advice is meant to satisfy. Complete accounting of all 328 decisions, a bounded review, and executable probes located a missed boundary defect and an alternative repair excluded by a narrower rubric. The released task corpus adds a structural reading of the domain: coded by failure loudness and blast radius, 28 of 61 incident tasks fail silently and nine misdirect — the classes that presence guards, crash logs, and aggregate scores all under-detect. Two blinded judges from other model families preserved the positive direction of the gain while changing its size. By keeping contracts, answers, scores, and counterexamples linked, the study makes both the observed benefit and its open correctness questions inspectable, and provides a reusable basis for the human validation, executable repairs, and cross-project replication outlined above.

Acknowledgments

We thank the other contributors to the open-source repository, including the GitHub users `whiteicey`, `AdamPlatin123`, and `HuanLinOTO`, for benchmark tasks, experiment runs, and migration cards.

Artifact Availability and Research Transparency

The working repository is <https://github.com/oh-my-dsh/dsh-plugin-upgrade-skill>. The focal archive is under `benchmark/results/artifacts/2026-09-15-glm-5.3-flash-unified-s16`; the supplementary review is under `paper/audit/output-review-20260917`. Original outputs and scores are preserved separately from review verdicts. Complete contract accounting and human-follow-up provenance are stored under `paper/audit/contract-review-20260917`. The cross-family judge panel, its protocol, and all panel verdicts are under `paper/audit/llm-judge-panel-v1`. Offline mechanism probes and their execution environment are archived under `paper/audit/mechanism-checks-20260917`. The claim-to-evidence ledger identifies inputs and offline commands for regenerating the analyses. Availability is described per configuration; no universal raw-output completeness claim is made.

Use of AI Assistance

AI tools (Codex and ChatGPT) assisted with source inspection, analysis code, initial targeted output review, and manuscript drafting. Non-blind human checking was subsequently performed by multiple contributing plugin authors. The archive preserves both stages and does not claim independently measured human agreement. In the cross-family judge panel, Claude Opus 5.5 (via Claude Code) and GPT-5.5 (via an isolated Codex CLI session) served as LLM judges; Claude Code also assisted with panel tooling and manuscript revision. Panel verdicts are LLM judgments and are reported as such.

References

- Aylton Almeida, Laerte Xavier, and Marco Tulio Valente. Automatic library migration using large language models: First results. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 427–433. ACM, 2024. doi: 10.1145/3674805.3690746.
- Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 1–10. ACM, 2011. doi: 10.1145/1985793.1985795.
- Christopher Bogart, Christian Kästner, James Herbsleb, and Ferdian Thung. How to break an API: cost negotiation and community values in three software ecosystems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 109–120. ACM, 2016. doi: 10.1145/2950290.2950325.
- Aline Brito, Laerte Xavier, Andre Hora, and Marco Tulio Valente. Why and how Java developers break APIs. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 255–265, 2018. doi: 10.1109/SANER.2018.8330214.
- Keyuan Cheng, Xudong Shen, Yihao Yang, Tengyue Wang, Yang Cao, Muhammad Asif Ali, Hanbin Wang, Lijie Hu, and Di Wang. CODEMENV: Benchmarking Large Language Models on Code Migration, 2025. URL <https://arxiv.org/abs/2506.00894v1>.
- Cheng-Han Chiang and Hung-yi Lee. Can large language models be an alternative to human evaluations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15607–15631, 2023. doi: 10.18653/v1/2023.acl-long.870.
- Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.
- Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968. doi: 10.1037/h0026256.
- Alexandre Decan and Tom Mens. What do package dependencies tell us about semantic versioning? *IEEE Transactions on Software Engineering*, 47(6):1226–1240, 2021. doi: 10.1109/TSE.2019.2918315.
- Danny Dig and Ralph Johnson. How do APIs evolve? A story of refactoring. *Journal of Software Maintenance and Evolution: Research and Practice*, 18(2):83–107, 2006. doi: 10.1002/smr.328.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993. doi: 10.1007/978-1-4899-4541-9.

- Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53. IEEE, 2023. doi: 10.1109/ICSE-FoSE59343.2023.00008.
- Shahriar Golchin and Mihai Surdeanu. Time Travel in LLMs: Tracing Data Contamination in Large Language Models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.08493>.
- Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on LLM-as-a-judge, 2024.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?, 2026. URL <https://arxiv.org/abs/2603.15401v1>.
- Junda He, Jieke Shi, Terry Yue Zhuo, Christoph Treude, Jiamou Sun, Zhenchang Xing, Xiaoning Du, and David Lo. From code to courtroom: LLMs as the new software judges, 2025.
- Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2024. doi: 10.1145/3695988.
- Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills, 2026. URL <https://arxiv.org/abs/2605.23899v1>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Raula Gaikovina Kula, Daniel M. German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. Do developers update their library dependencies? *Empirical Software Engineering*, 23(1):384–417, 2018. doi: 10.1007/s10664-017-9521-5.
- Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. A systematic review of API evolution literature. *ACM Computing Surveys*, 54(8):171:1–171:36, 2022. doi: 10.1145/3470133.
- J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977. doi: 10.2307/2529310.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-chung Lee. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks, 2026. URL <https://arxiv.org/abs/2602.12670v1>.

- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.03688>.
- Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, 2023. doi: 10.18653/v1/2023.emnlp-main.153.
- Tyler McDonnell, Baishakhi Ray, and Miryung Kim. An empirical study of API stability and adoption in the Android ecosystem. In *2013 IEEE International Conference on Software Maintenance (ICSM)*, pages 70–79, 2013. doi: 10.1109/ICSM.2013.18.
- Ian R. McKenzie, Alexander Lyzhov, Michael Pieler, Alicia Parrish, Aaron Mueller, Ameya Prabhu, Euan McLean, Aaron Kirtland, Alexis Ross, Alisa Liu, et al. Inverse Scaling: When Bigger Isn’t Better. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=DwgRm72GQF>.
- Stoyan Nikolov, Daniele Codecasa, Anna Sjövall, Maxim Tabachnyk, Satish Chandra, Siddharth Taneja, and Celal Ziftci. How is Google using AI for internal code migrations? In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 481–492, 2025. doi: 10.1109/ICSE-SEIP66354.2025.00048.
- Justin D. Norman, Michael U. Rivera, and D. Alex Hughes. Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias, 2026. URL <https://arxiv.org/abs/2606.19544v1>.
- Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL <https://arxiv.org/abs/2404.13076>.
- Steven Raemaekers, Arie van Deursen, and Joost Visser. Semantic versioning and impact of breaking changes in the Maven repository. *Journal of Systems and Software*, 129:140–158, 2017. doi: 10.1016/j.jss.2016.04.008.
- Paul Ralph et al. Empirical standards for software engineering research, 2020.
- Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi: 10.1007/s10664-008-9102-8.
- June Sallou, Thomas Durieux, and Annibale Panichella. Breaking the silence: the threats of using LLMs in software engineering. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pages 102–106. ACM, 2024. doi: 10.1145/3639476.3639764.
- Klaas-Jan Stol and Brian Fitzgerald. The ABC of software engineering research. *ACM Transactions on Software Engineering and Methodology*, 27(3):1–51, 2018. doi: 10.1145/3241743.
- Pat Verga, Sebastian Hofstatter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models, 2024.

- Stefan Wagner, Marvin Muñoz Barón, Davide Falessi, and Sebastian Baltes. Towards evaluation guidelines for empirical studies involving LLMs. In *2025 IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering (WSESE)*, pages 24–27. IEEE, 2025. doi: 10.1109/WSESE66602.2025.00011.
- Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9440–9450, 2024. doi: 10.18653/v1/2024.acl-long.511.
- Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. Can LLMs replace human evaluators? an empirical study of LLM-as-a-judge in software engineering. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1955–1977, 2025a. doi: 10.1145/3728963.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025b. URL <https://openreview.net/forum?id=OJd3ayDDoF>.
- Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.
- Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012. doi: 10.1007/978-3-642-29044-2.
- Tongtong Wu, Weigang Wu, Xingyu Wang, Kang Xu, Suyu Ma, Bo Jiang, Ping Yang, Zhenchang Xing, Yuan-Fang Li, and Gholamreza Haffari. VersiCode: Towards Version-controllable Code Generation, 2024. URL <https://arxiv.org/abs/2406.07411v2>.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754.
- Shengzhe Xu, Ziqi Dong, and Na Meng. Meditor: Inference and application of API migration edits. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, pages 335–346, 2019. doi: 10.1109/ICPC.2019.00052.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024. doi: 10.52202/079017-1601.
- Ziyue Yang and Fan Ding. Signal or Noise? A Benchmark Study of Agent Skills in Web Development, 2026. URL <https://arxiv.org/abs/2608.23067v1>.
- Barry Zhang, Keith Lazuka, and Mahesh Murag. Equipping agents for the real world with agent skills. Anthropic Engineering Blog, 2025. Published October 16, 2025. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, accessed 2026-09-24.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2306.05685>.

Terry Yue Zhuo. ICE-Score: Instructing large language models to evaluate code. In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 2232–2242, 2024. doi: 10.18653/v1/2024.findings-eacl.148.

Celal Ziftci, Stoyan Nikolov, Anna Sjövall, Bo Kim, Daniele Codecasa, and Max Kim. Migrating code at scale with LLMs at Google. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, pages 162–173. ACM, 2025. doi: 10.1145/3696630.3728542.

A Task-level focal results

Table 5: Focal task-level original scores. Each arm has two repeats; differences use arm means. These are static rubric scores, not repair pass rates.

Task	No skill r1	No skill r2	Skill r1	Skill r2	Mean Δ
S1	55.00	60.00	100.00	100.00	42.50
S2	100.00	80.00	100.00	100.00	10.00
S5	87.50	100.00	100.00	100.00	6.25
S6	87.50	87.50	100.00	100.00	12.50
S9	100.00	100.00	100.00	100.00	0.00
S11	100.00	90.00	90.00	90.00	-5.00
S12	100.00	100.00	100.00	100.00	0.00
S13	100.00	100.00	100.00	100.00	0.00
S14	100.00	100.00	100.00	100.00	0.00
S15	100.00	100.00	100.00	100.00	0.00
S17	80.00	80.00	90.00	100.00	15.00
S18	100.00	100.00	100.00	90.00	-5.00
S19	100.00	100.00	100.00	100.00	0.00
S20	97.50	97.50	100.00	100.00	2.50
S21	100.00	100.00	100.00	100.00	0.00
S22	100.00	100.00	100.00	100.00	0.00

B Failure-mode taxonomy per-task coding

Per-task coding behind Table 4; regenerated by `paper/scripts/generate-failure-taxonomy.mjs` (-check mode wired into CI), which also asserts that every `benchmark/tasks` entry carries a coding.

Task	Loudness	Radius	Basis
H1-plane-trap	misdirected	local	source comments steer toward a fatal change
H2-baseline-trap	misdirected	local	pre-existing red test predates the migration
H3-client-plane	loud	local	missing required declaration refuses load
H4-tsbuildinfo-trap	misdirected	local	stale build artifact implicates deleted API
H5-runtime-export-drift	silent	local	green locally, drifts at runtime
H6-remote-error-trap	misdirected	local	comment pins old error handling
H7-locale-trap	silent	local	display-text anchoring breaks silently
H8-fire-drill	misdirected	cotenant	three plugins, three trap states in one release
H9-dsh-web-alpha2	loud	local	real-repo migration, import surface fails loudly

Task	Loudness	Radius	Basis
H10-browser-activation-trap	silent	local	manifest lists it, bundle 200s, never activates
H11-dual-cohort-rpc	silent	local	passes mocks, fails the real host
H12-remote-result-boundary-trap	silent	local	error vocabulary right, control flow wrong
H13-ghost-host-trap	silent	system	disk upgraded, host process still old
H14-mineru-api	loud	local	channel 404s after apiPrefix change
H15-locale-pack	silent	local	monkey-patch target moves silently
H16-history-dock	silent	local	contenteditable needs capture-phase listener
H17-merge-calls	loud	local	deleted derivations fail imports
H18-blame-bubbles	loud	local	deleted facade fails the call
H19-workspace-ya	silent	cotenant	official disabled UI must be taken over via slots
H20-session-events-ledger	loud	local	removed getter throws on iteration
H21-question-answerer-waterfall	silent	local	single-seat registration silently degrades
H22-dsh-data-agent-alpha2	loud	local	real-repo migration, loud import breaks
H23-storage-domain-version-compat-trap	silent	local	domain bump silently opens old data
H24-invalid-record-salvage-trap	misdirected	local	salvage logic implicates the wrong record
H25-session-seed-boundary-trap	silent	local	seed boundary semantics drift
H26-notlisted-trap	silent	local	installs cleanly, never registers
H27-undeclared-import-trap	loud	local	ERR_MODULE_NOT_FOUND at load
M1-host-migration	loud	local	plugin fails to start on new host
M2-optional-dep-trap	loud	local	unconditional import of optional dep
M3-session-projection	silent	local	degraded behavior without declared projection service
M4-peer-prerelease-range	silent	local	semver prerelease exclusion installs clean
M5-token-auth-smoke	silent	local	channel works, host auth silently missing
M6-sleep-tool	silent	local	tool shape drifts silently
M7-d399-overlay	loud	local	inject names deleted runtime, boot-fatal
M8-brand-text	loud	local	moved store engine and stale peer
M9-mcpanel	loud	local	removed runtime in inject, boot-fatal
M10-tools-tree	silent	local	service moved, missing silently
M11-sidebar-spur	loud	local	Context import from deleted runtime
M12-interpreters-card	silent	local	renamed Settings service unresolved
M13-repository-plugins-removal	loud	local	removed plugin shape never loads
M14-service-renames-0812	loud	local	pre-0812 service names fail resolution
S1-static-scan	procedural	—	coverage scan exercise, no single incident
S2-negative-scan	procedural	—	negative control, no incident

Task	Loudness	Radius	Basis
S3-snapshot-migration	silent	local	legacy snapshot reads degrade without errors
S4-legacy-client-imports	loud	local	module resolution fails at build/boot
S5-negative-naming	procedural	—	negative control, no incident
S6-corridor-net-state	silent	local	intermediate-state defense mis-handles new state
S7-unpublished-cohort	silent	local	packaging trap: installs locally, absent on registry
S8-release-routing-trap	misdirected	local	install failure blames mirror; cause is tag routing
S9-composer-coordinate-trap	silent	local	first use works, later ones silently misbehave
S10-paste-rename-and-version-chip	procedural	—	post-release follow-up procedures
S11-mermaid-lazyload-trap	loud	local	chunk 404 and window errors at load
S12-global-upgrade-ebusy-trap	misdirected	system	EBUSY/MSVC errors blame locks; cause is live-process self-upgrade
S13-peer-range-vs-runtime	loud	local	installs clean, crashes at runtime start
S14-link-install-lock-trap	silent	local	link edits silently never reach the GUI
S15-slot-error-boundary-crash	loud	cotenant	one entry’s throw unmounts the whole shared dock
S16-self-host-upgrade-trap	misdirected	system	npm replaces files under the running host itself
S17-external-ui-plugin-onboarding-trap	loud	local	raw-ESM bundle fails the module loader
S18-terminal-sprite-render-trap	silent	local	phantom pixels, no error
S19-phantom-update-stale-host	silent	local	update badge misleads about currency
S20-msvc-flock-trap	loud	system	native build dies, environment-level
S21-resource-service-unavailable-trap	silent	local	document tab opens but content silently fails
S22-duplicate-insert-boot-crash-trap	loud	system	duplicate loader id crashes boot
S23-passing-compat-guard-trap	silent	local	field-level removal passes presence guards
S24-dock-cotenant-crash-trap	loud	cotenant	shared drawer boundary unmounts innocent co-tenant

C Historical and supplementary context

C.1 Historical paired summaries

For each historical configuration, we use its recorded task reward on a 0–100 scale and preserve its original within-task aggregation. Qwen uses means of three scored attempts, DeepSeek and the two GLM groups use medians of three runs or rounds, and Terra uses one attempt. These numeric scales do not imply equivalent

Model (n tasks)	Tasks	no-skill $\rightarrow$ with-skill	Mean paired Δ	95% CI	Signed- rank p	Protocol
qwen3.8-27b ($n = 56$)	full 56-task pool	44.67 $\rightarrow$ 41.60	-3.07	$[-8.13, +1.99]$	0.1771	3 scored, mean
deepseek-v4-flash ($n = 23$)	23-task snapshot	69.96 $\rightarrow$ 80.63	$+10.67$	$[+0.48,$ $+22.11]$	0.1007	3-run median
gpt-5.6-terra ($n = 21$)	22-task pool, 21 scored	71.10 $\rightarrow$ 79.76	$+8.67$	$[-4.29,$ $+21.62]$	0.2359	single-shot
glm-5.3-flash ($n = 22$)	S1–S22 static	77.77 $\rightarrow$ 87.05	$+9.27$	$[+3.95,$ $+16.09]$	0.0056	3-round median
glm-5.2 ($n = 22$)	S1–S22 static	93.32 $\rightarrow$ 96.36	$+3.05$	$[0.00, +7.18]$	0.1378	3-round median

Judge identity: in both GLM groups the solver and the grader belong to the same model family: glm-5.3-flash graded its own runs, and 19 of the 22 glm-5.2 tasks were judged by glm-5.3-flash subagents (the remaining three by official keyword judges), so correlated grader bias cannot be excluded (Section 2). glm-5.2’s CI lower bound touching zero alongside $p = 0.1378$ is a percentile-bootstrap boundary artifact: with 16 of 22 task deltas tied at zero, the lower endpoint of the resampling distribution sits at zero, so the interval brushing zero does not contradict the nonsignificant test.

Table 7: Task-level paired effect of the plugin-upgrade skill across five historical configurations, ordered by each group’s no-skill baseline (an outcome measure, not an independent capability metric; Section C.1). Each cell compares the two conditions on a 0–100 scale: per-task medians of three rounds (glm groups) or three runs (deepseek-v4-flash), per-task means of three scored attempts (qwen3.8-27b, reward means rescaled by 100), or single-shot rewards (gpt-5.6-terra; H8 excluded after verifier timeouts on both arms). Mean paired Δ is the mean of per-task with-skill-minus-no-skill deltas; 95% CIs are percentile intervals from 10,000 task-level paired bootstrap replicates (mulberry32, seed 20260907); p is the two-sided Wilcoxon signed-rank test (zero deltas excluded, tie-corrected normal approximation with continuity correction). Task pools and protocols differ across rows, so cross-row comparisons are descriptive.

constructs: keyword grading, semantic judgment, and composite executable rewards differ. We do not pool configurations or regress their gains on their baseline scores as an independent capability measure.

Historical percentile intervals use 10,000 task-paired bootstrap samples with mulberry32 seed 20260907. The reported two-sided Wilcoxon signed-rank calculations exclude zero differences and use a tie-corrected normal approximation with continuity correction. These exploratory summaries can disagree near a boundary; neither is a substitute for validating the endpoint. No multiple-comparison-adjusted confirmatory family is claimed.

Historical mean changes range from -3.07 to $+10.67$ points (Table 7). The older Qwen configuration has substantial skill-arm timeouts under its local infrastructure. Terra excludes H8 because both arms have verifier timeouts. The GLM groups mix semantic and keyword judgments. These differences prohibit attributing cross-row variation to model capability alone. A contaminated Luna baseline, which accessed a native plugin-creator skill on H2 and H3, is excluded from this five-configuration comparison.

Within the historical GLM pair, the Flash-minus-5.2 difference in mean task lift is 6.23 points, with a paired bootstrap interval $[2.59, 10.05]$. The round-specific Flash gains are 6.32, 8.82, and 9.82, versus 1.55, 1.64, and 5.00 for GLM-5.2. Leaving one task out yields lift gaps from 5.33 to 7.00. These completed robustness calculations support a descriptive contrast within the recorded protocols. Different judge mixes and development histories remain confounders; the contrast does not identify a capability-dependent response curve.

C.2 Newer supplementary configurations

A supplementary newer Qwen run on the same S16 configuration reports 71.17 to 77.89, a gain of 6.72 points, with a reported interval $[-12.42, 26.25]$. It contains 64 scored attempts, including five timeouts and two other nonzero-exit attempts. Termination status is distinct from the assigned score. Its raw answers and grading reasons are not archived, and the reported bootstrap randomization is insufficiently specified for exact interval reproduction. Fourteen tasks overlap with the focal Flash run; different hosts, budgets, judges, and selections prevent a controlled cross-model comparison. We use this result only as supplementary context.

Three supplementary GLM-5.3 rounds on S1–S22 are now archived. Recomputing all 132 verdicts against the pinned rubric, retaining half-point values before summation, gives round gains of 1.93, 3.18, and 6.59 points. The per-task median across rounds is 97.05 without the skill versus 98.64 with it, a gain of 1.59 points; its descriptive task-bootstrap interval is $[-1.36, 5.80]$ (10,000 samples, seed 20260907). The third round used GLM-5.3 as judge rather than the GLM-5.3-Flash judge used in rounds one and two. The median cannot remove this protocol change, so these results remain a mixed-judge descriptive supplement.

Round three’s S17 no-skill answer receives zero because its affirmative immediate-registration recommendation triggers the rubric’s contradiction cap. The criterion decisions are retained: this pair contributes 100 of the round’s 145 total gain points. Excluding S17 descriptively leaves a 2.14-point mean gain across 21 tasks; this is a sensitivity, not a basis for removing the observation. The archive discloses quota-related relaunches, concurrency changes, and a truncated round-one answer. These rounds do not establish a capability ladder or restore comparability with older protocols.

C.3 Historical resource accounting

We also recomputed the earlier Flash round-one usage totals from its 22 sessions per condition. Fresh input totals are 599,882 and 678,101; output totals are 116,506 and 132,726; cache totals are 5,890,176 and 5,017,984. The recorded total-token sums are 6,606,564 and 5,828,811, respectively. These fields do not support an undifferentiated claim of a 2.2-fold token increase. Their accounting differs from the focal host’s token field, so resource ratios are reported within configurations only.

D Judge panel provenance

The panel protocol, analysis plan, blinded items, unblinding key, and all verdicts are under `paper/audit/llm-judge-panel-v1`; `prepare-llm-judge-panel.mjs` and `analyze-llm-judge-panel.mjs` in `paper/scripts` regenerate the items and results and check them for drift. The protocol was written before any panel verdict existed but was not separately committed or externally registered beforehand. The Claude session that prepared the items had seen earlier results; it judged no item and passed no outcome information to the 16 judging sessions. The operator of the GPT-5.5 session had seen manuscript context before reading the blinding protocol and passed no outcomes to the isolated judge. Both judges’ metadata files record these facts, including incidental directory-name listings in which no verdict or key file was opened. GPT-5.5’s single sequential session may carry context between items, unlike the original per-report calls.

E Artifact map and reproduction scope

Historical result files are stored under `benchmark/results`: `paired-effect-stats.json` generates the paired table, and `glm-pair-stability.json` records GLM stability. The focal directory contains the selection, schedule, execution-order records, original reports and judgments, aggregate scores, paired analysis, and the earlier targeted review. The later review stores frozen selection and complete criterion

verdicts separately. The script `paper/scripts/summarize-submission-evidence.mjs` checks report hashes, recomputes review scores, deduplicates overlapping answers, and regenerates sensitivity and historical resource summaries without model calls. Its `-check` mode verifies the committed generated summary.

A frozen 23-task historical inventory and its source metadata are retained in `paper/generated`; they are not substituted for the 16-task selection. Exposure information remains in `paper/audit/task-exposure-ledger.csv`. Missing raw artifacts, unverified model identities, and pending author declarations are recorded as limitations rather than filled in by inference.